YEI 3-Space Sensor C API Quick-Start Guide

Purpose

The purpose of this document is to introduce users to the YEI 3-Space Sensor C Application Programming Interface (API). After reading this document, users will be capable of instantiating the API's various classes, interacting with the 3-Space family of sensors, and interpreting any read data in a meaningful way.

Overview

The 3-Space C API is a collection of convenience functions that wrap all normal functionality of Yost Engineering Inc's 3-Space wired sensors, wireless sensors, and wireless dongles for use in a program written in C/C++ or any language that can import Microsoft Windows dynamic link libraries (.dll).

Software Requirements

Operating System – Windows XP 32-Bit, Windows Vista 32-Bit/64-Bit, or Windows 7 32-Bit/64-Bit (Windows XP 64-Bit is untested)

Getting Started With The API

When connecting to and communicating with a piece of 3-Space hardware via the C API, there are a few common steps that must be taken before full communication may take place. These steps are as follows:

1. Link to DLL and include API header
2. Scan for Available 3-Space Hardware
3. Construct Interface Instances of 3-Space hardware

This document is structured such that steps 2 and 3 are discussed first. If guidance is required for linking the dll to the user's development environment, please refer to the **Link to DLL and Include API Header** section at the end of the Quick-Start.

Basic code examples for constructing Interface Instances

These examples and more are on the YEI 3-Space website

Creating Class Instances

This example finds and creates the device and returns a TSS_ID which is needed to call many functions in the API

```
#include "yei_threespace_api.h"
#include <stdio.h>

int main(){
```

If the COM port is already known and the device type is known for the 3-Space Sensor device, we can just create the appropriate instance without doing a search.

```
	TSS_ID  device;
	char hex_str[9];
	printf("====Creating TS Devices on COM71 ====\n");
	device = createTSDeviceStr("COM71", TSS_WL);
	if( device == NO_DEVICE_ID){
		printf("Could not create device\n");
	}
	else{
		getSerialNumberHex(device,hex_str);
		printf("Port created successfully for sensor %s\n", hex_str);
	}
	closeTSDevice(device);
```

If the COM port is not known or the device type is not known for the 3-Space Sensor device, we must do a search for the devices. We can do this by calling the getAvailableComPorts function which returns two lists of COM port information.(known 3-Space Sensor devices and unknown devices) getAvailableComPorts also as a parameter called filter_list that takes a list of 3-Space Sensor device types which can be used to find specific 3-Space Sensor devices. If filter_list is not used or set to None all connected 3-Space Sensor devices are found. Each COM port information is a list containing(COM port name, friendly name, 3-Space Sensor device type)

```
	ComPort *tss_ports;
	ComPort *ukn_ports;
	unsigned int tss_size;
	unsigned int ukn_size;

	getAvailableComPorts(	&tss_ports,
					&ukn_ports,
					&tss_size,
					&ukn_size,
					TSS_FIND_ALL);
	printf("====Creating Known TS Devices ====\n");
	for( unsigned int i=0; i< tss_size; ++i){
		device =createTSDevice(tss_ports[i]);
```

Check if the id was successfully returned, unless the device is in use by another application this usually succeeds

```
		if( device != NO_DEVICE_ID){
			getSerialNumberHex(device,hex_str);
			printf("Port created successfully for sensor %s\n", hex_str);
			closeTSDevice(device);
		}
		else{
			printf("Failed to create a sensor on %s\n",tss_ports[i].com_port);
		}
	}
```

The unknown list could contain 3-Space Sensor devices that were connected via a RS232 connection. We can find out if there is any 3-Space Sensor devices by using the getComPortInfo function. However, we must send commands over the port which could make other devices other than 3-Space Sensor devices act in an undesirable way. So be sure to know what port your 3-Space Sensor device(s) are on when calling this function.
getComPortInfo returns a list of information that it found from the COM port(Friendly name, 3-Space Type, 3-Space ID, 3-Space Firmware Version String, 3-Space Hardware Version String, is in bootloader mode)

```
	printf("====Poking Unknown Ports For TS Devices====\n");
	for( unsigned int i=0; i< ukn_size; ++i){
		ComInfo com_info;
		getComPortInfo(ukn_ports[i].com_port, &com_info, true);
		if( com_info.sensor_type != TSS_UNKNOWN){
			device =createTSDeviceStr(ukn_ports[i].com_port,
com_info.sensor_type);
			getSerialNumberHex(device,hex_str);
			printf("Port created successfully for sensor %s\n", hex_str);
			closeTSDevice(device);
		}
		else{
			printf("Sensor Not found\n");
		}
	}
```

This an alternate method of getting sensors, this one you call getFirstAvailableComPort and all the rest are called getNextAvailableComPort. If the call fails TSS_ERROR_NO_SENSOR_FOUND is returned

```
	printf("===========Using getFirstAvailableComPort Method=============\n");
	ComPort comport;
	if(!getFirstAvailableTSSComPort(&comport, TSS_FIND_ALL)){
		device =createTSDevice(comport);
```

Check if the id was successfully returned, unless the device is in use by another application this usually succeeds

```
		if( device != NO_DEVICE_ID){
			getSerialNumberHex(device,hex_str);
			printf("Port created successfully for sensor %s\n", hex_str);
			closeTSDevice(device);
		}
		else{
			printf("Failed to create a sensor on %s\n",comport.com_port);
		}
		while( !getNextAvailableTSSComPort(&comport)){
			device =createTSDevice(comport);
			if( device != NO_DEVICE_ID){
				getSerialNumberHex(device,hex_str);
				printf("Port created successfully for sensor %s\n", hex_str);
				closeTSDevice(device);
			}
			else{
				printf("Failed to create a sensor on %s\n",comport.com_port);
			}
		}
	}

	printf("Finished press Enter to continue");
	getchar();
	return 0;
}
```

Getting Information

This example demonstrates getting basic data from the sensor. This is compatible with all sensors plugged in via USB or Bluetooth. Will not work with the dongle or wireless sensor wirelessly see Getting and Setting Wireless Information Devices for a wireless example. For Sensors plugged in with RS232 see Creating Class Instances

```
#include "yei_threespace_api.h"
#include <stdio.h>

int main(){
	TSS_ID  device;
	TSS_Error tss_error;
	ComPort comport;

	if(!getFirstAvailableTSSComPort(&comport, TSS_FIND_ALL^TSS_FIND_DNG)){
		device =createTSDevice(comport);
		if( device == NO_DEVICE_ID){
			printf("Failed to create a sensor on %s\n",comport.com_port);
			return 1;
		}
	}
	else{
		printf("No sensors found\n");
		return 1;
	}
	printf("=================================================\n");
	printf("Getting the filtered tared quaternion orientation.(xyzw)\n");
```

Many functions provide a structure or an array to get the data

```
	TSS_Quaternion quat;
	tss_error= getFiltTaredOrientQuat(device, &quat);
	if(!tss_error){
		printf("Quaternion: %f, %f, %f, %f\n", quat.x, quat.y, quat.z ,quat.w);
	}
	else{
		printf("TSS_Error: %s\n", TSS_Error_String[tss_error]);
	}
	printf("=================================================\n");
	printf("Getting the raw sensor data.\n");
	float raw_data[9];
	tss_error= getAllSensorsRawf9(device, raw_data);
	if(!tss_error){
		printf("Gyro:  %f, %f, %f\n", raw_data[0],raw_data[1],raw_data[2]);
		printf("Accel: %f, %f, %f\n", raw_data[3],raw_data[4],raw_data[5]);
		printf("Comp:  %f, %f, %f\n", raw_data[6],raw_data[7],raw_data[8]);
	}
	else{
		printf("TSS_Error: %s\n", TSS_Error_String[tss_error]);
	}
	printf("=================================================\n");
	printf("Getting the LED color of the device.\n");
	TSS_Color color;
	tss_error= getLEDColor(device, &color);
	if(!tss_error){
		printf("Color: %f, %f, %f\n", color.r, color.g, color.b);
	}
	else{
		printf("TSS_Error: %s\n", TSS_Error_String[tss_error]);
	}
	printf("=================================================\n");
```

```
	closeTSDevice(device);

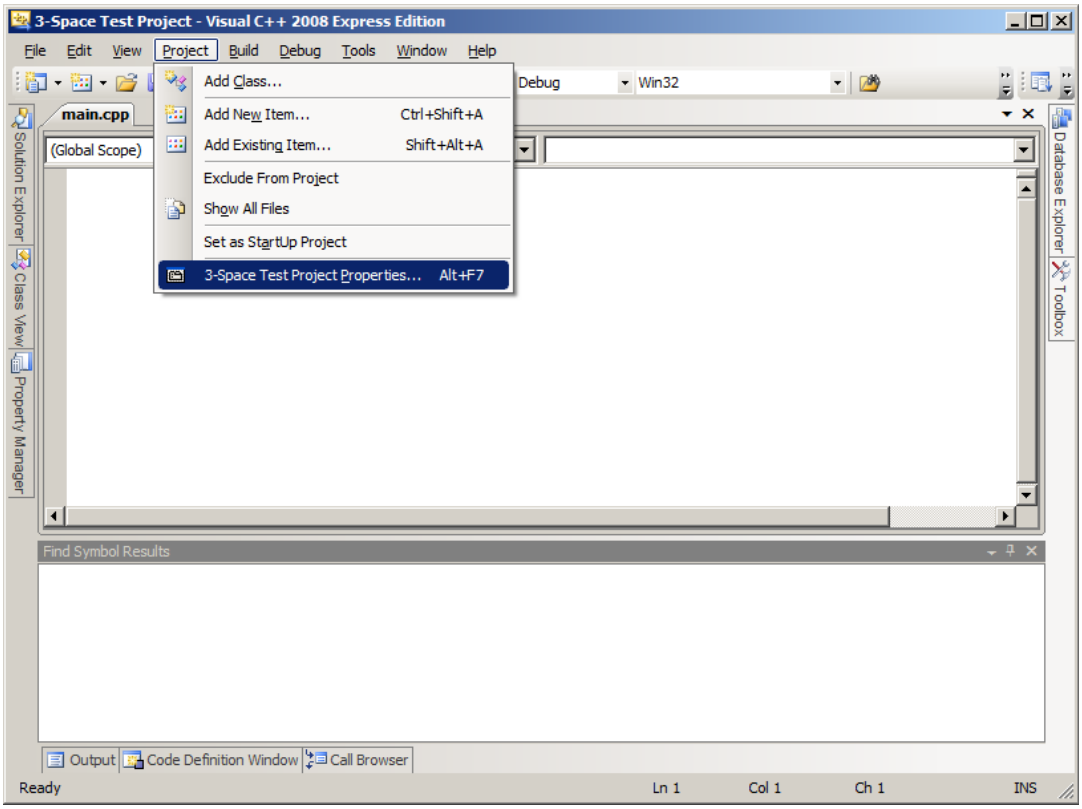
	printf("Finished press Enter to continue");
	getchar();
	return 0;
}
```

Link to DLL and Include API Header

This step configures the programmer's development environment to link to the included dynamic link library and includes the API header in the programmer's project. To use the 3-Space C API, programmers must configure their C/C++ project to include the dll packaged with the API download at runtime. The process for doing this differs slightly depending on the development environment. The three cases covered in this guide are users using Microsoft Visual Studio 2008, users using Microsoft Visual Studio 2010, and users using CodeBlocks. After downloading the C API files to a known location, follow the steps below based on the development environment being used:

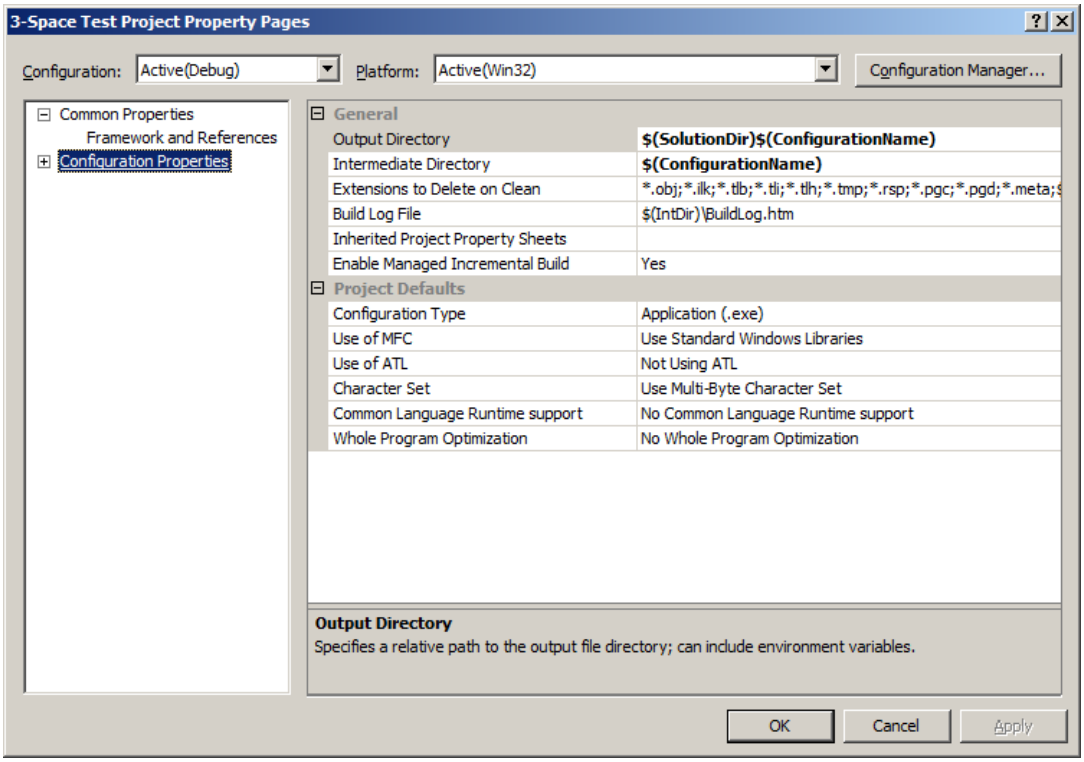
Microsoft Visual Studio 2008

Assuming a project has already been created, we must first open the projects properties dialog. This can be found clicking on “Project” → “*name_of_project* Properties...”. This menu can be found near the top of the Visual Studio window.



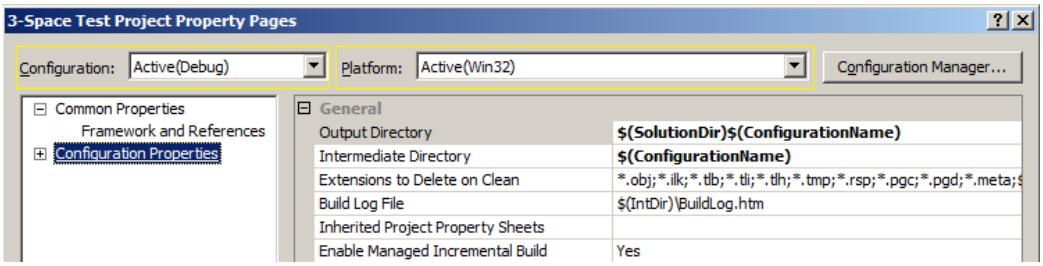
The name of this sample project is “3-Space Test Project”, so the menu item selected is “Project → 3-Space Test Project Properties”.

Once the project properties dialog, one must ensure the changes made to the project's settings take effect for any project configuration. To do this, first select the “Configuration Properties” title in the left column of the prompt.



The Project Properties dialog. Notice how the "Configuration Properties" title is selected.

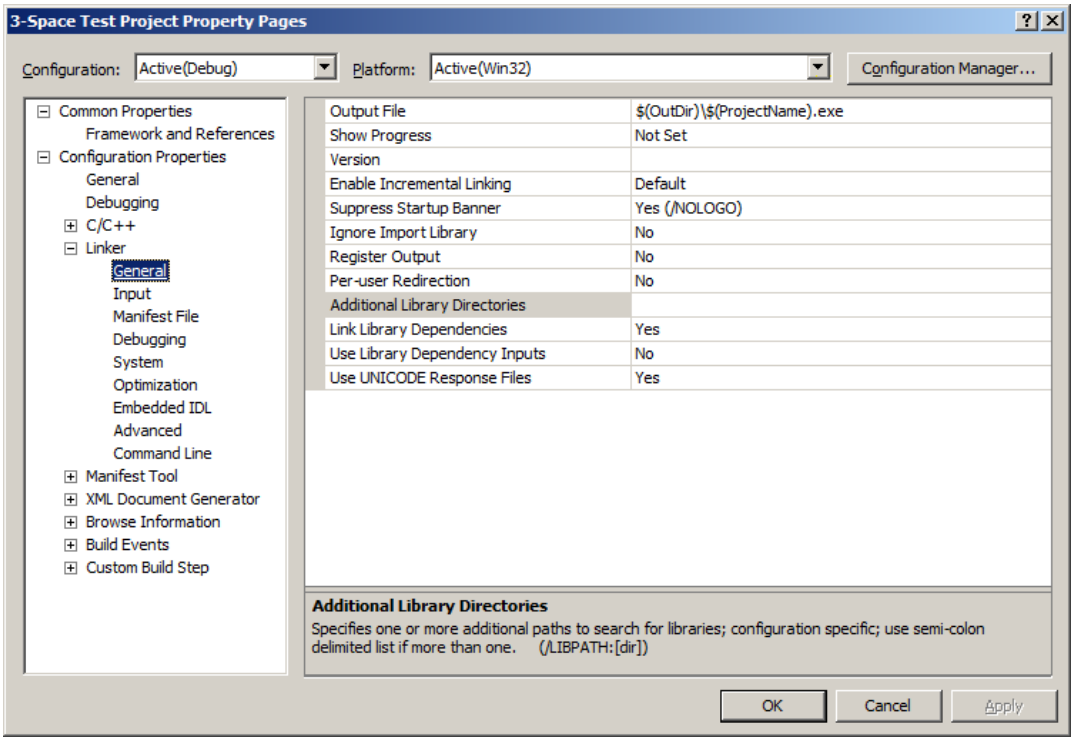
Clicking this title will make two drop-down selection boxes at the top of the prompt become active. They are labeled "Configuration" and "Platform".



The top of the project properties dialog with the "Configuration" and "Platform" drop-down boxes active. The boxes have been highlighted.

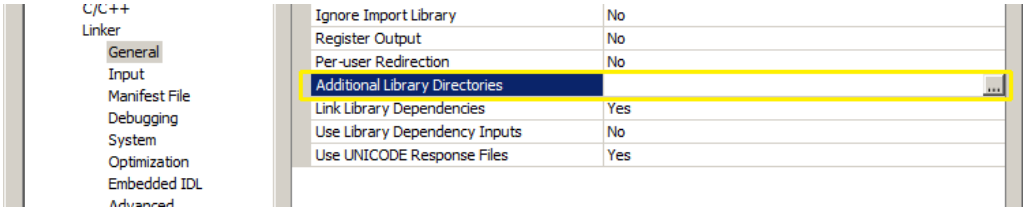
The Configuration box and Platform box are used to list the different compilation/linking profiles for the project. Project properties changed in the prompt are reflected only in the compilation/linking profile expressed in these boxes. As such, it is wise to ensure the project properties for each of the profiles being used are correctly set.
The Configuration box also has an "All Configurations" option as well. Properties changed with this option selected will be changed in all profile permutations created by the Configuration box options.

Once the desired project profile is selected, next expand the "Configuration Properties" title in the left column by clicking the "+" sign next to it. Once expanded, find the "Linker" title listed underneath it. Expand this title as well and select the "General" title under the "Linker" title.

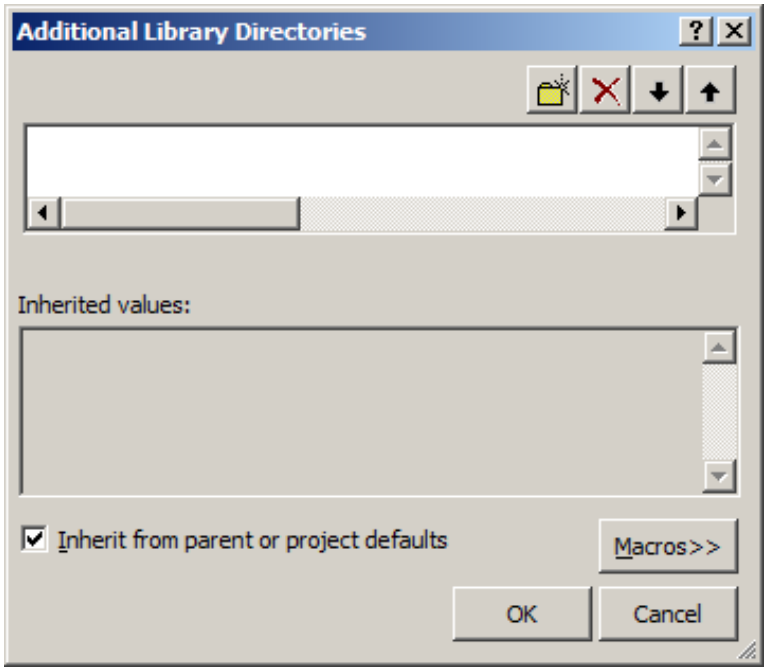


The project properties dialog with the "General" title of the "Linker" title selected.

In the area to the right of the "title column" is a listing of properties for the selected title. The property of interest for this configuration is the "Additional Library Directories" property. This property defines additional file directory to search through for .lib files. Said files are usually paired with dynamic link library files (.dll) and tell the linker how to resolve function names to areas of the dll. Click on the empty area to the right of the "Additional Library Directories" title to make a button appear to modify this property. This button will have three dots "..." on it. When clicked, a new prompt will be shown that will allow the user to search their file system for the directory to add.

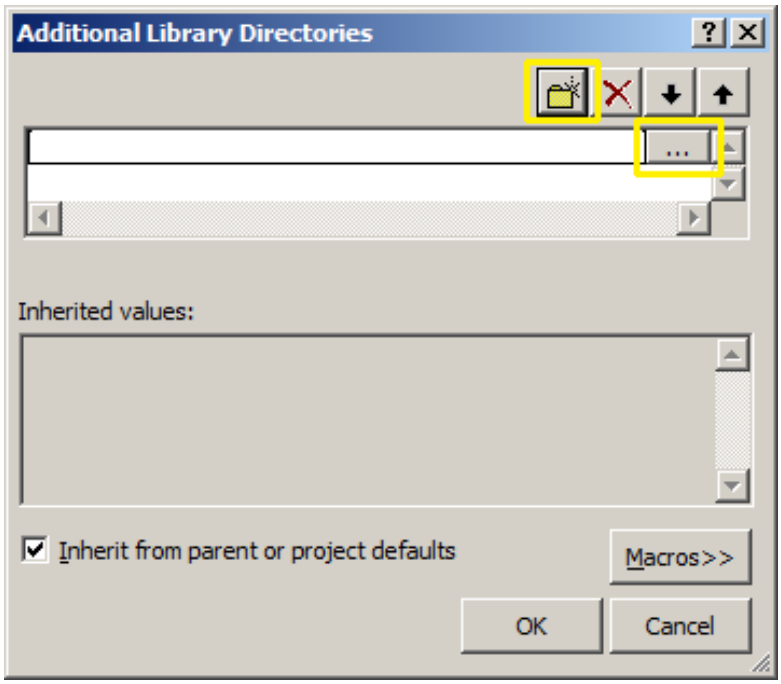


The "Additional Library Directories" property is selected and has been highlighted. Please note the "..." button on the far right of the blank area.

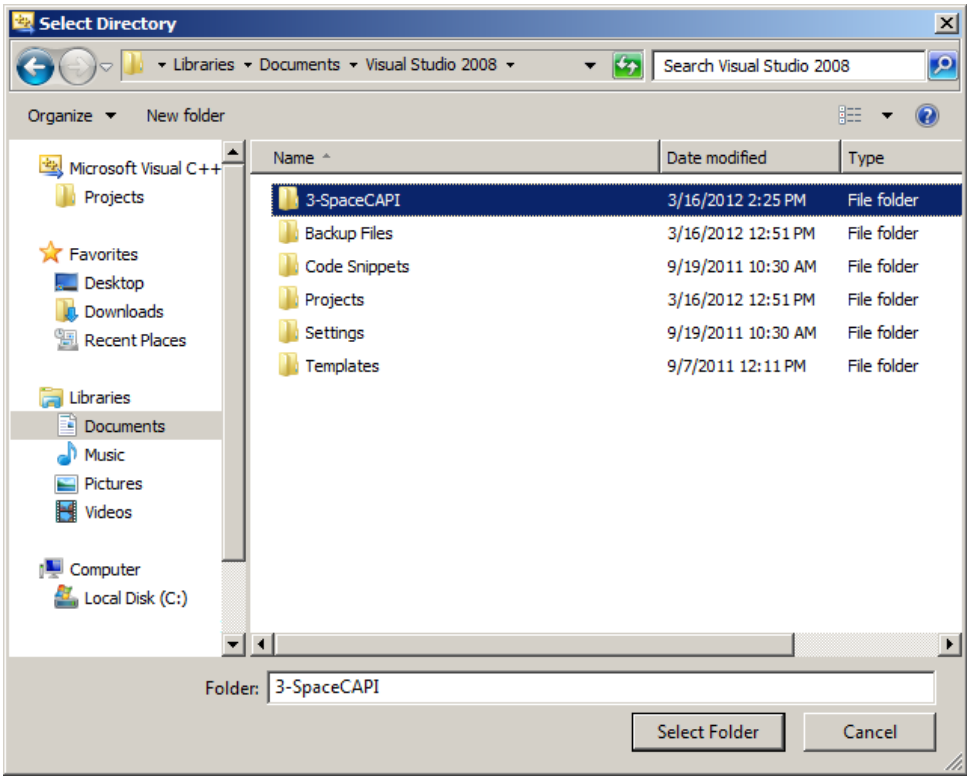


The prompt displayed when the "..." button is pressed.

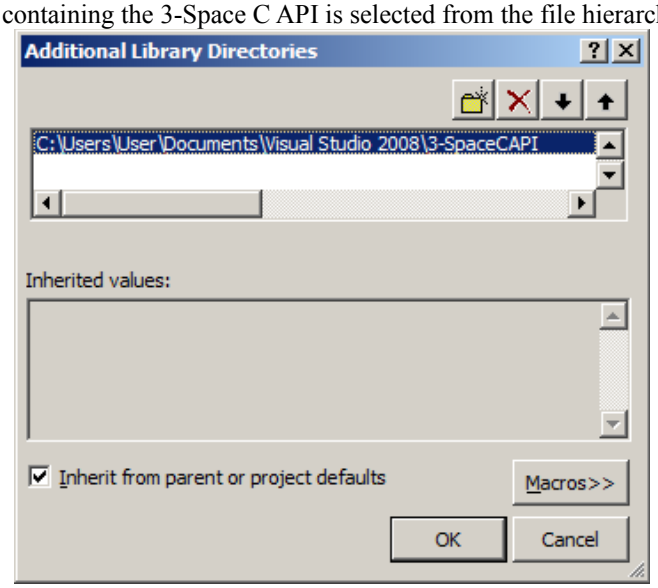
With the "Additional Library Directories" prompt open, click on the left-most button in the group of four buttons in the upper right area of the prompt (the button with a folder on it). This is the "new line" button. This will add a new directory to the blank area with the white background. This area lists all additional directories. Once pressed, the newly added directory will have another "..." button to the right of it. Click this button to open a file hierarchy browser prompt. Using this prompt, find the directory that stores the 3-Space C API files and add it.



With the "new line" button pressed, a new directory listing has been added. Both the "new line" and "..." buttons have been highlighted.



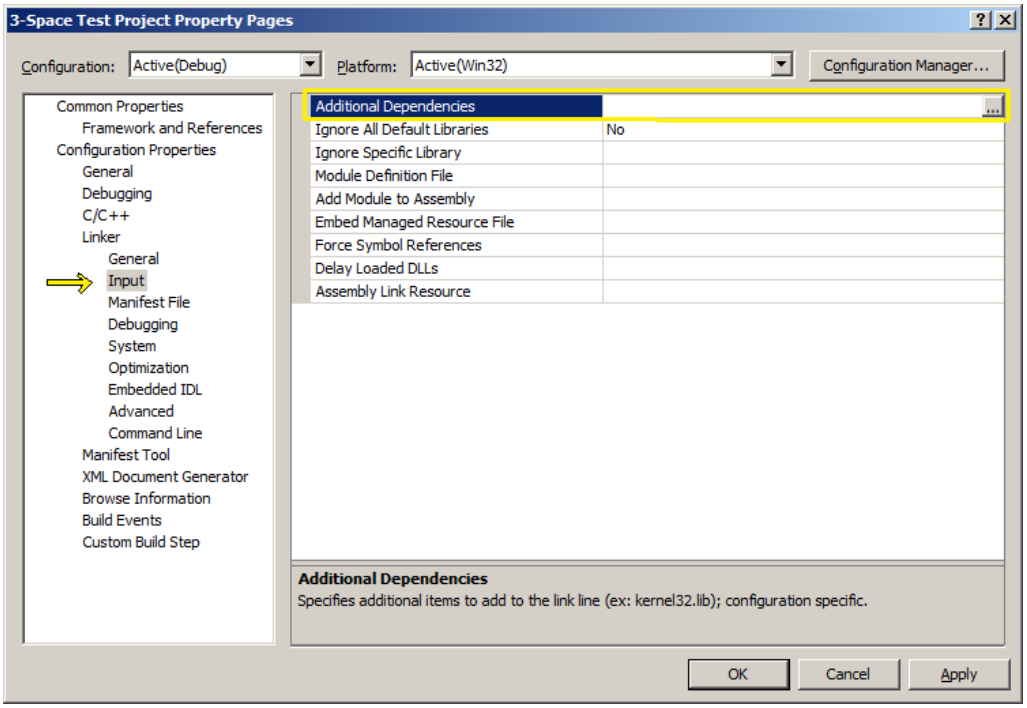
The folder containing the 3-Space C API is selected from the file hierarchy browser.



The “Additional Library Directories” prompt with the newly added directory.

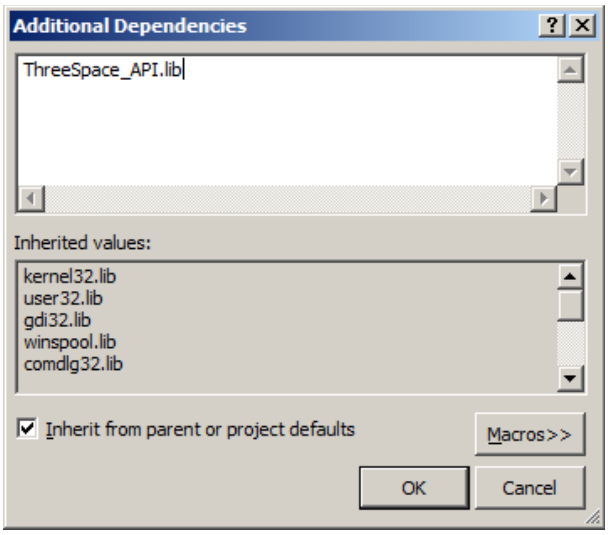
Once the folder has been added, press the “OK” button in the prompt to close it. Next the .lib file itself

must be added to the project. To do this, from the project properties dialog click on the “Input” title grouped under the “Linker” title. Once this title has been selected, click on the “Additional Dependencies” property in the right-hand area of the prompt. Once selected, click the “...” button that appears in the blank area to the right of the property's title.



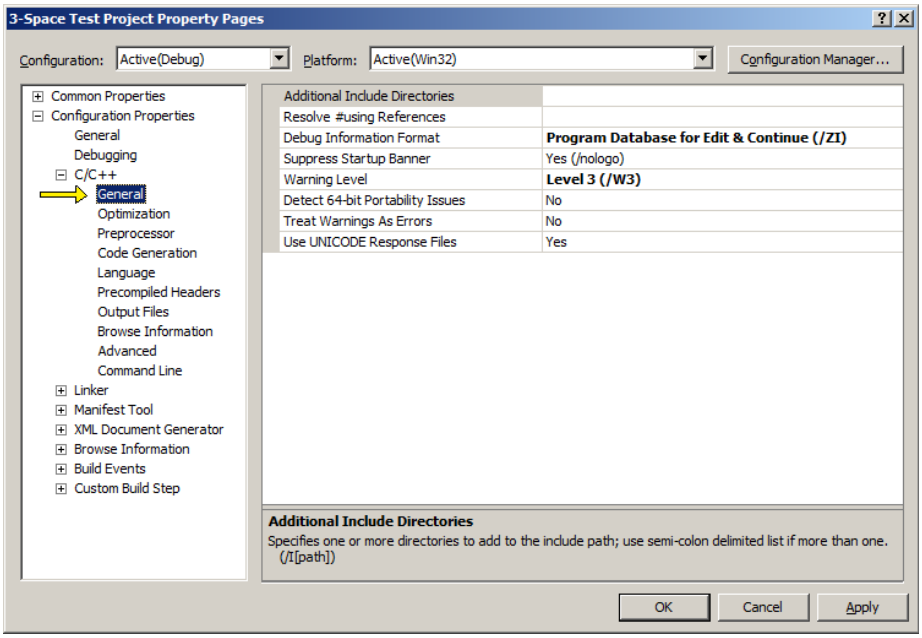
The project properties prompt with the “Additional Dependencies” property selected. Please note how the “Input” title has been selected and the “...” button is present to the right of the property.

Once the "..." button is pressed, the "Additional Dependencies" prompt will be displayed. This prompt is used to list all .lib files the linker needs to look for when the compiled program is linked. The .lib packaged with the 3-Space API will be added to this list. The top area of this prompt is user editable an is where we will be adding the .lib. To add the .lib, simply type *ThreeSpace_API.lib* into this area and press the "OK" button.



The "Additional Dependencies" prompt. Please note that *Threespace_API.lib* is typed into the upper area of the prompt.

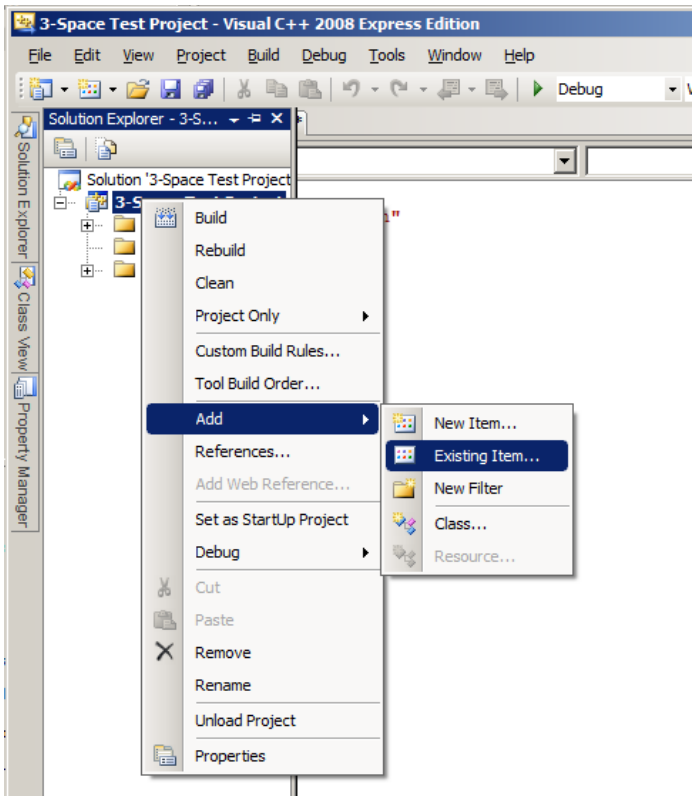
The final configuration that needs done in the project properties dialog is to tell Visual Studio where to find header files (.h) not in the immediate folder structure of the project. To do this, first collapse the "Linker" title by clicking the "-" symbol next to the title. Once collapsed, expand the "C/C++" title that is under the "Configuration Properties" title by clicking the "+" next to the "C/C++" title. From the titles listed under the "C/C++" title, select the "General" title.



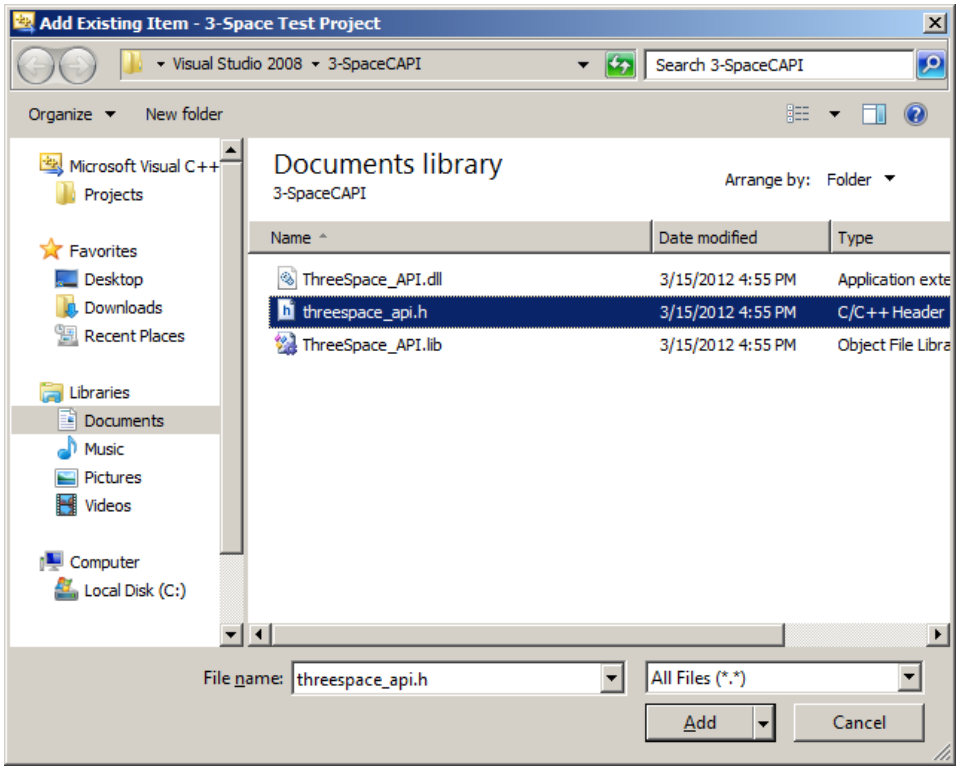
The project properties dialog with the "Configuration Properties → C/C++ → General" title selected.

With “General” selected, Select the “Additional Include Directories” property in the right-hand area of the dialog. Once selected, a “...” button will appear. Click this button to open a prompt similar to the “Additional Library Directories” prompt. Also much like that prompt, use the “new line” button to add a new listing for a directory to look for header files in. Follow the same steps as listed for the “Additional Library Directories” portion of this guide to find the folder containing *threespace_api.h* and add it to the prompt's listing of directories. Once added, press the “OK” button to close the “Additional Include Directories” prompt.

Once this is done, press the “Apply” button in the project properties prompt. Once pressed, press the “OK” button to close the prompt.
With the .lib now added to the project, the next step is to include the 3-Space API header in the project. This process is done by first going to the “Solution Explorer” in Visual Studio, right-clicking on the project in the explorer, then selecting Add → Existing Item. When selected, a file hierarchy browser will open. From this browser, find the folder that the downloaded API files were saved to. Once this folder is open, select the *threespace_api.h* file from the folder. Click the “Add” button to add this file to the project.



With the Solution Explorer open, the project has been right-clicked and the option to add an existing item has been selected.



The file hierarchy browser window with *threespace_api.h* selected.

Once added, include the header to the current project by writing the following line to the top of the appropriate source file:

```
#include "threespace_api.h"
```

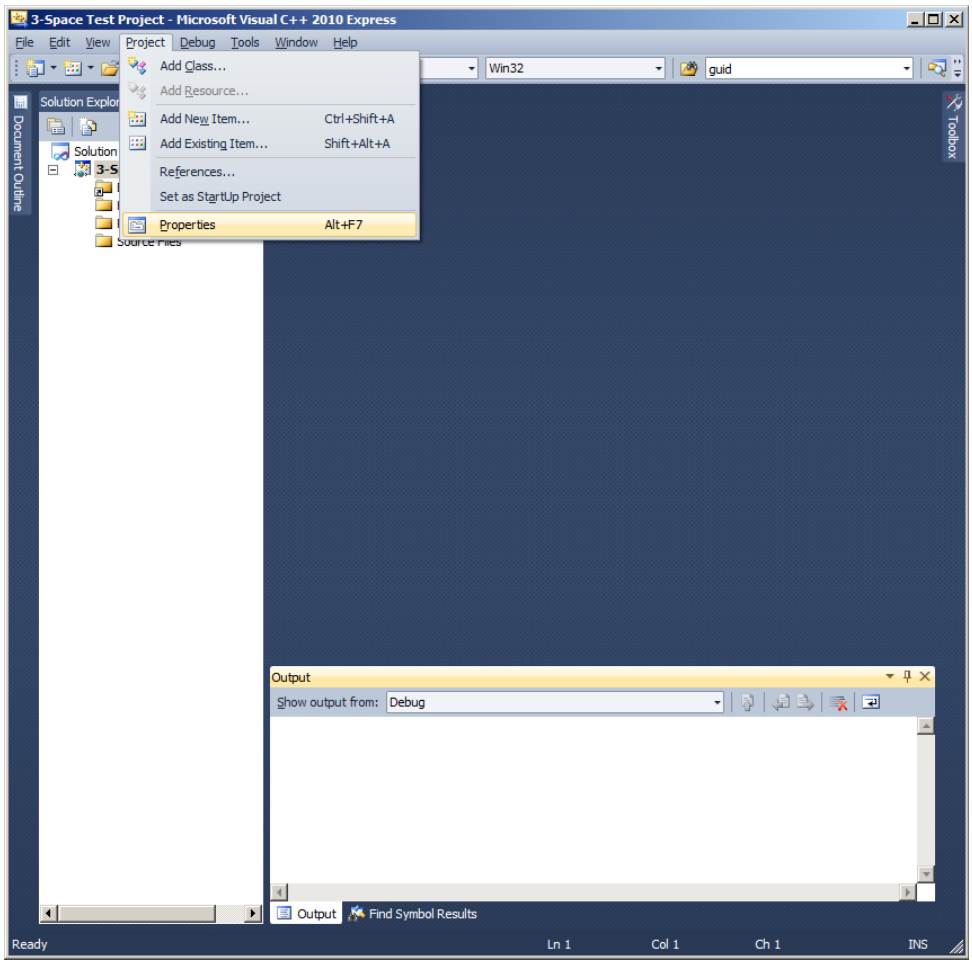
Once written, the program should compile and link fine. The last consideration to make is to ensure the *threespace_API.dll* file is in the correct folder. Proper locations for the .dll are either:

1. A folder listed in the operating system "path".
2. The same directory as the built binary program created by Visual Studio.

With this final consideration made, programs may now be written and executed that interact with the YEI 3-Space family of sensors. Included with the API is the test source file *threespace_test.cpp*. Import this source file into an empty Visual Studio project and follow the steps detailed above to integrate the 3-Space API. Once done, build and run the program to ensure the project is configured correctly.

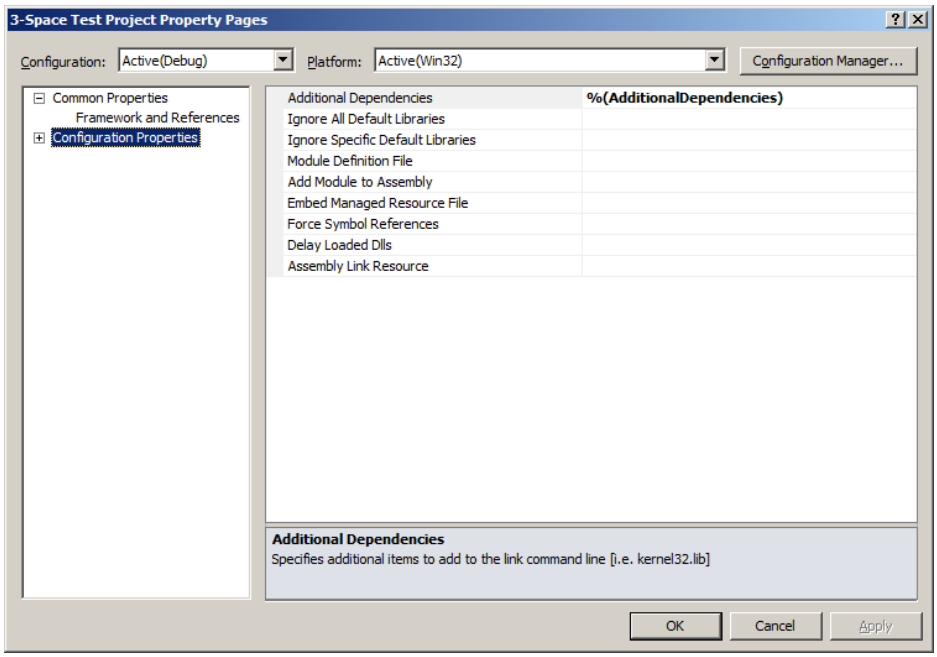
Microsoft Visual Studio 2010

Assuming a project has already been created, we must first open the projects properties dialog. This can be found clicking on “Project → Properties...”. This menu can be found near the top of the Visual Studio window.



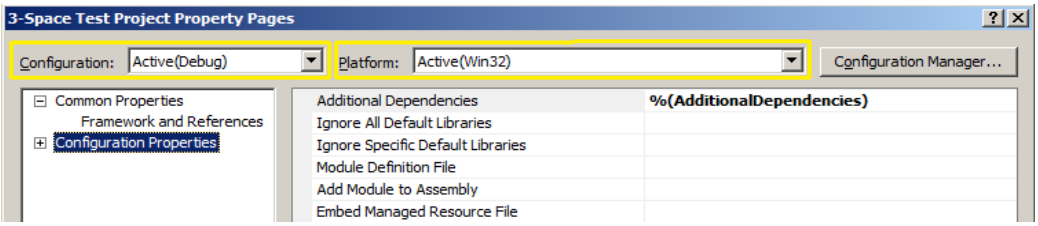
The main Visual Studio view with the project properties menu being selected.

Once the project properties dialog, one must ensure the changes made to the project's settings take effect for any project configuration. To do this, first select the “Configuration Properties” title in the left column of the prompt.



The Project Properties dialog. Notice how the “Configuration Properties” title is selected.

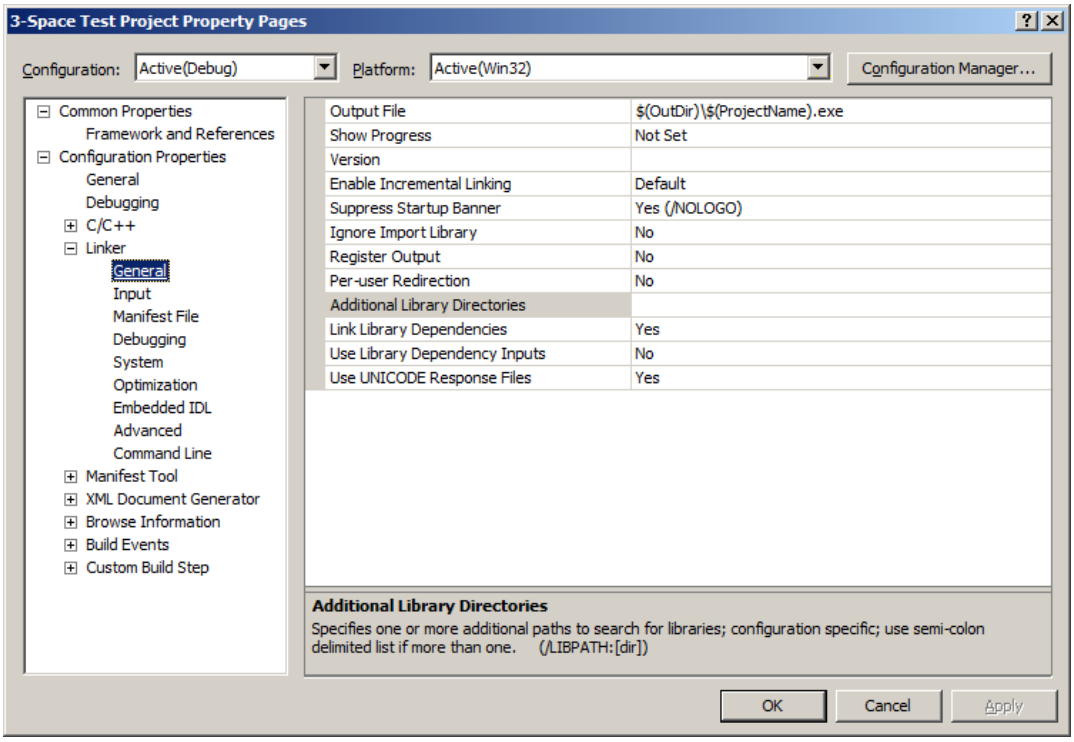
Clicking this title will make two drop-down selection boxes at the top of the prompt become active. They are labeled “Configuration” and “Platform”.



The top of the project properties dialog with the “Configuration” and “Platform” drop-down boxes active. The boxes have been highlighted.

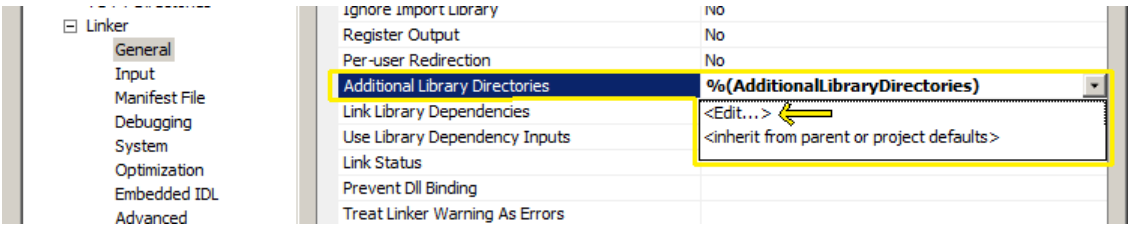
The Configuration box and Platform box are used to list the different compilation/linking profiles for the project. Project properties changed in the prompt are reflected only in the compilation/linking profile expressed in these boxes. As such, it is wise to ensure the project properties for each of the profiles being used are correctly set.
The Configuration box also has an “All Configurations” option as well. Properties changed with this option selected will be changed in all profile permutations created by the Configuration box options.

Once the desired project profile is selected, next expand the “Configuration Properties” title in the left column by clicking the “+” sign next to it. Once expanded, find the “Linker” title listed underneath it. Expand this title as well and select the “General” title under the “Linker” title.

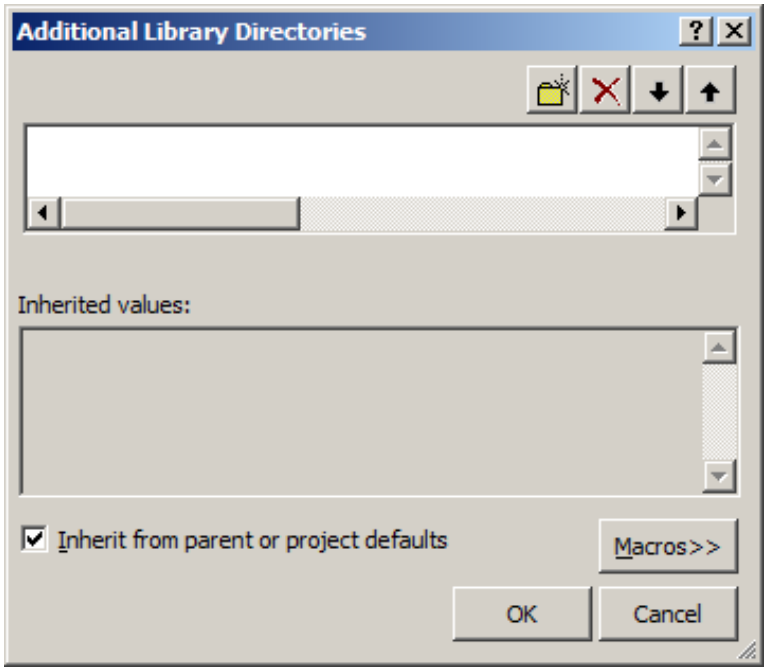


The project properties dialog with the "General" title of the "Linker" title selected.

In the area to the right of the "title column" is a listing of properties for the selected title. The property of interest for this configuration is the "Additional Library Directories" property. This property defines additional file directory to search through for .lib files. Said files are usually paired with dynamic link library files (.dll) and tell the linker how to resolve function names to areas of the dll. Click on the empty area to the right of the "Additional Library Directories" title to make a button appear to modify this property. This button will have an arrow on it. When clicked, a drop-down menu will appear listing two options; "<Edit...>" and "<inherit from parent or project defaults>". Select "<Edit...>". When selected, a new prompt will be shown that will allow the user to search their file system for the directory to add.

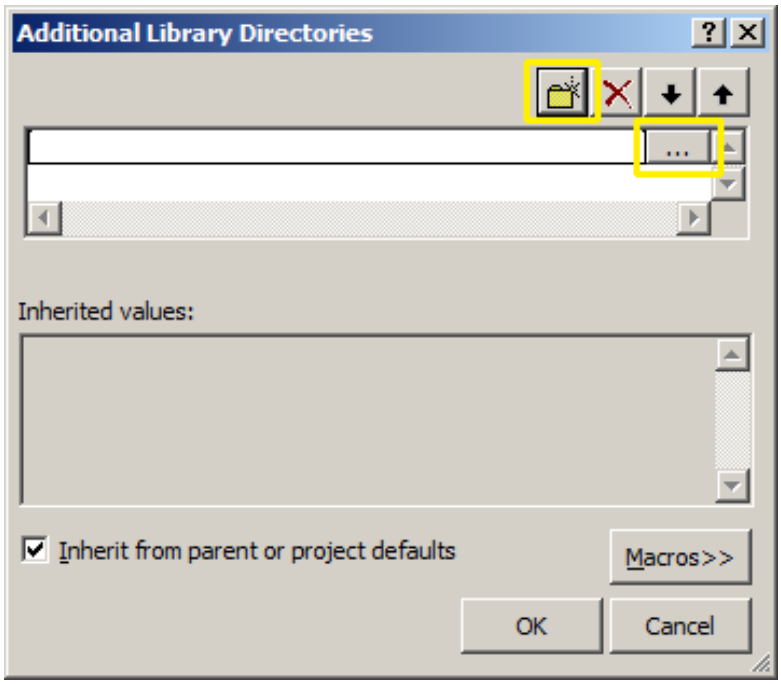


The "Additional Library Directories" property is selected and has been highlighted. Please note that the "<Edit...>" option is also highlighted. This is the choice to click.

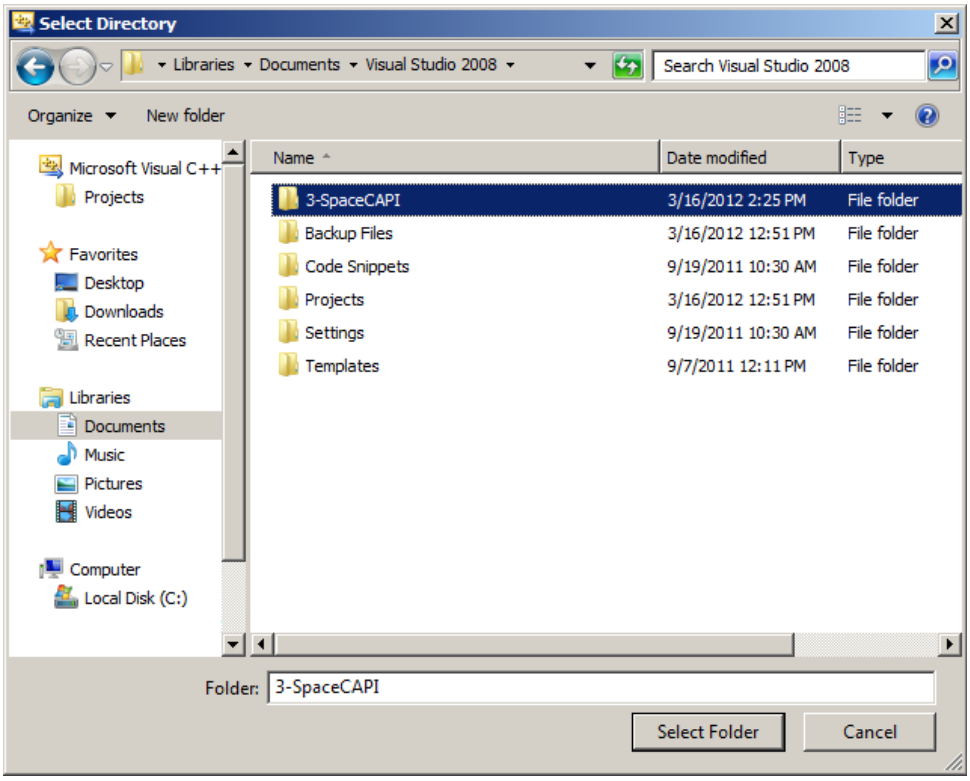


The prompt displayed when the "..." button is pressed.

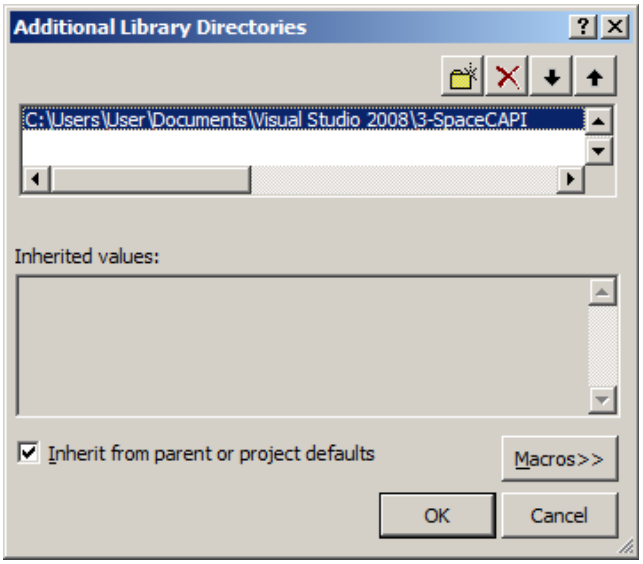
With the "Additional Library Directories" prompt open, click on the left-most button in the group of four buttons in the upper right area of the prompt (the button with a folder on it). This is the "new line" button. This will add a new directory to the blank area with the white background. This area lists all additional directories. Once pressed, the newly added directory will have another "..." button to the right of it. Click this button to open a file hierarchy browser prompt. Using this prompt, find the directory that stores the 3-Space C API files and add it.



With the "new line" button pressed, a new directory listing has been added. Both the "new line" and "..." buttons have been highlighted.

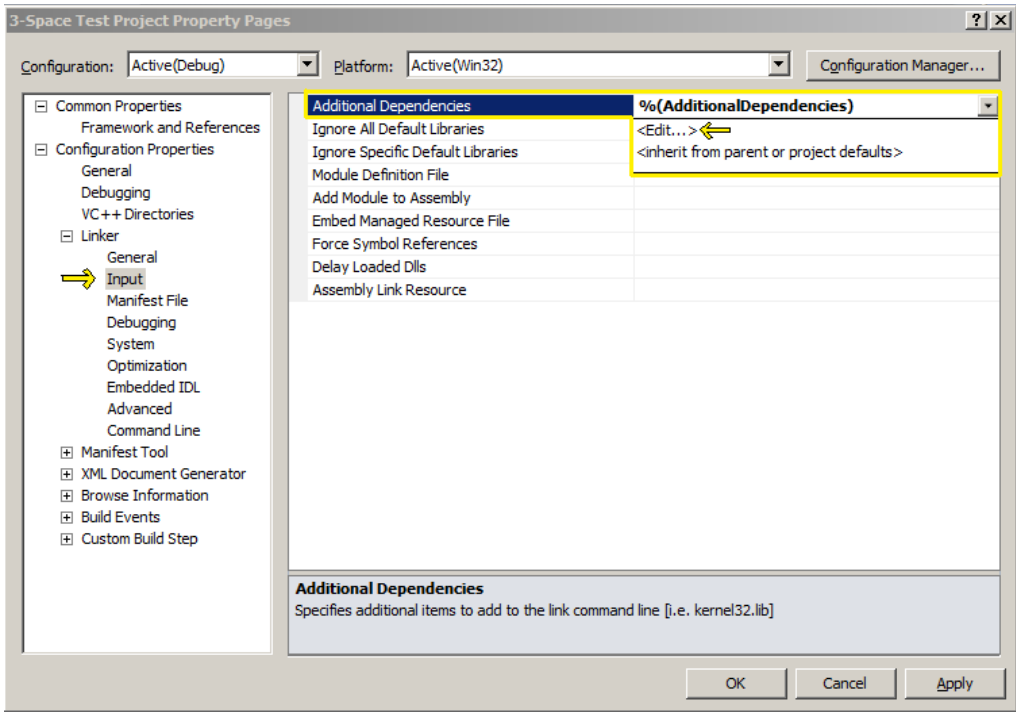


The folder containing the 3-Space C API is selected from the file hierarchy browser.



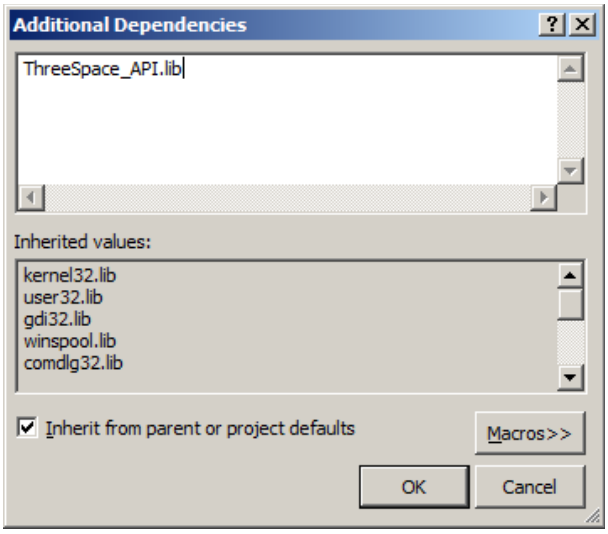
The “Additional Library Directories” prompt with the newly added directory.

Once the folder has been added, press the “OK” button in the prompt to close it. Next the .lib file itself must be added to the project. To do this, from the project properties dialog click on the “Input” title grouped under the “Linker” title. Once this title has been selected, click on the “Additional Dependencies” property in the right-hand area of the prompt. Once selected, click the drop-down button and select “<Edit...>” just like for the “Additional Library Directories” property.



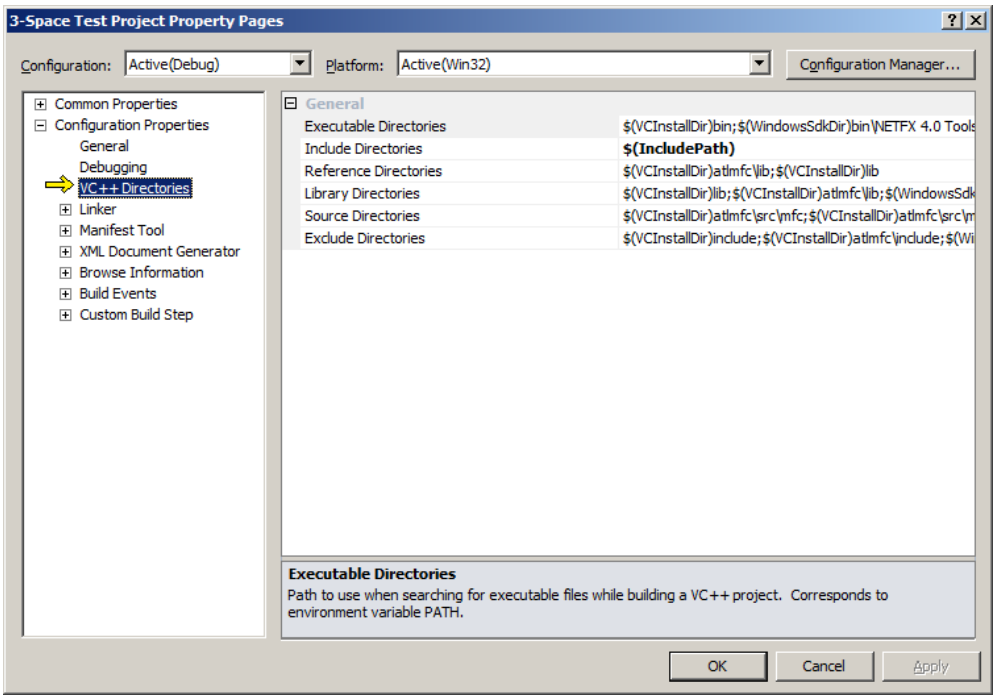
The project properties prompt with the “Additional Dependencies” property selected. Please note how the “Input” title has been selected and the “<Edit...> selection is highlighted.

Once “<Edit...> is selected, the “Additional Dependencies” prompt will be displayed. This prompt is used to list all .lib files the linker needs to look for when the compiled program is linked. The .lib packaged with the 3-Space API will be added to this list. The top area of this prompt is user editable an is where we will be adding the .lib. To add the .lib, simply type *ThreeSpace_API.lib* into this area and press the “OK” button.



The “Additional Dependencies” prompt. Please note that *Threespace_API.lib* is typed into the upper area of the prompt.

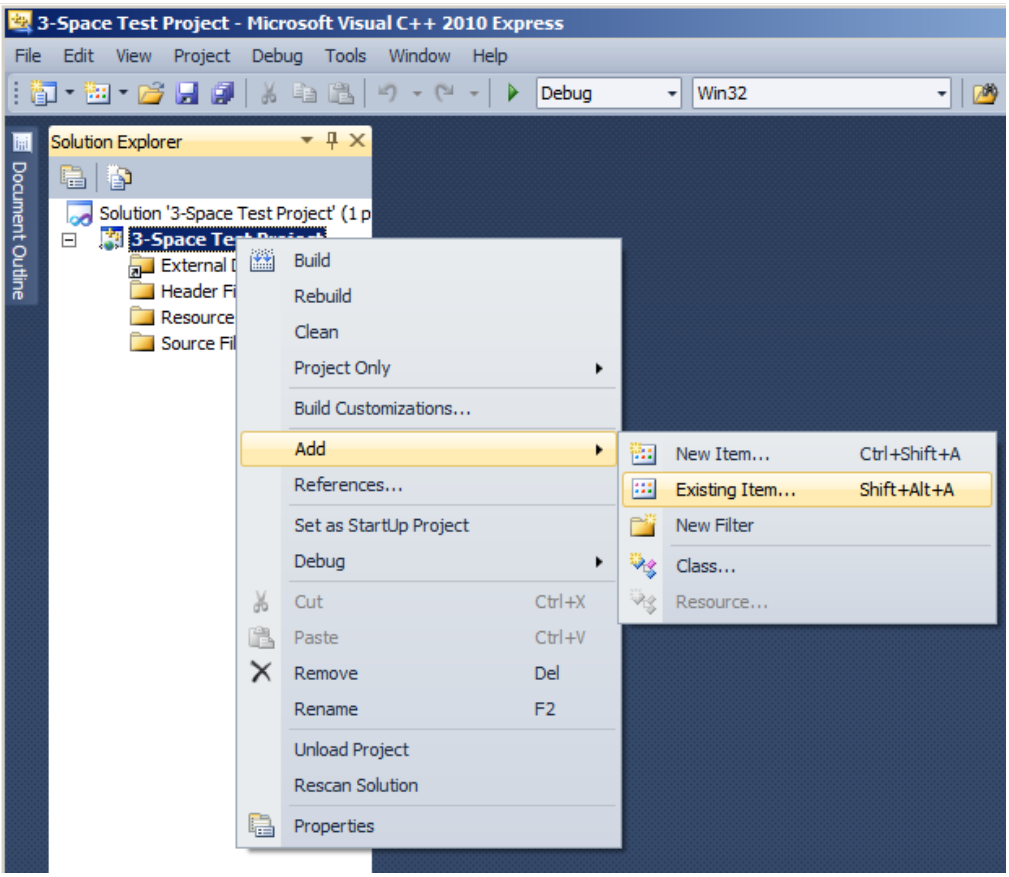
The final configuration that needs done in the project properties dialog is to tell Visual Studio where to find header files (.h) not in the immediate folder structure of the project. To do this, first collapse the “Linker” title by clicking the “-” symbol next to the title. Once collapsed, click on the “VC++ Directories” title.



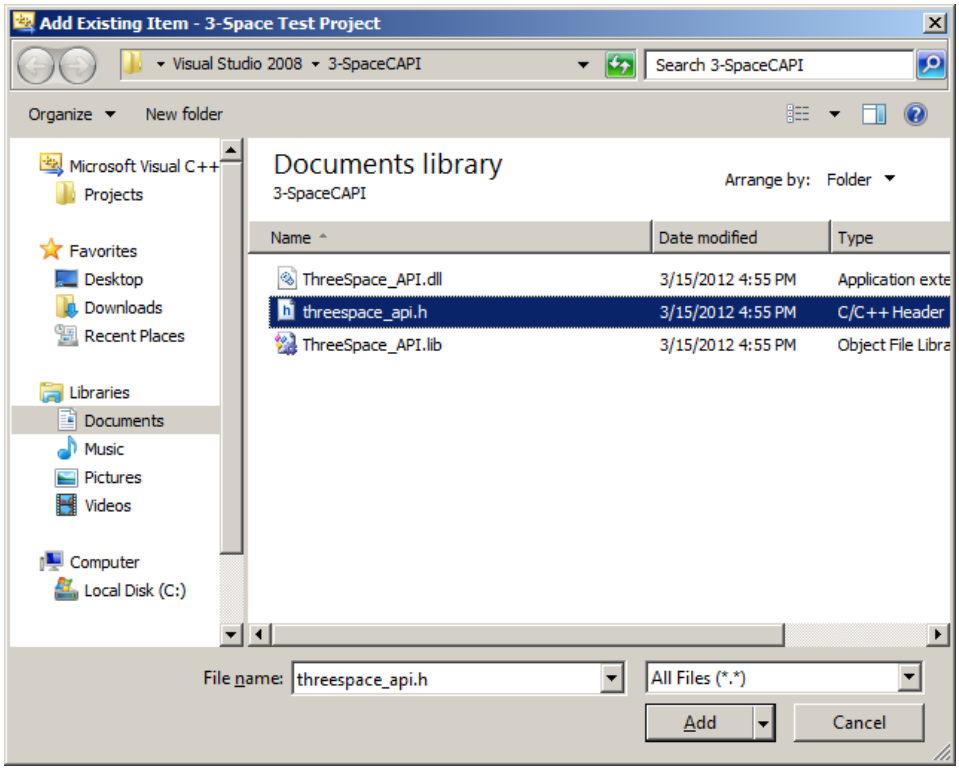
The project properties dialog with the “VC++ Directories” title selected.

With the title selected, Select the “Include Directories” property in the right-hand area of the dialog. Once selected, a drop-down button will appear. Click this button and select “<Edit...>” to open a prompt similar to the “Additional Library Directories” prompt. Also much like that prompt, use the “new line” button to add a new listing for a directory to look for header files in. Follow the same steps as listed for the “Additional Library Directories” portion of this guide to find the folder containing *threespace_api.h* and add it to the prompt's listing of directories. Once added, press the “OK” button to close the “Additional Include Directories” prompt.

Once this is done, press the “Apply” button in the project properties prompt. Once pressed, press the “OK” button to close the prompt.
With the .lib now added to the project, the next step is to include the 3-Space API header in the project. This process is done by first going to the “Solution Explorer” in Visual Studio, right-clicking on the project in the explorer, then selecting Add → Existing Item. When selected, a file hierarchy browser will open. From this browser, find the folder that the downloaded API files were saved to. Once this folder is open, select the *threespace_api.h* file from the folder. Click the “Add” button to add this file to the project.



With the Solution Explorer open, the project has been right-clicked and the option to add an existing item has been selected.



The file hierarchy browser window with *threespace_api.h* selected.

Once added, include the header to the current project by writing the following line to the top of the appropriate source file:

```
#include "threespace_api.h"
```

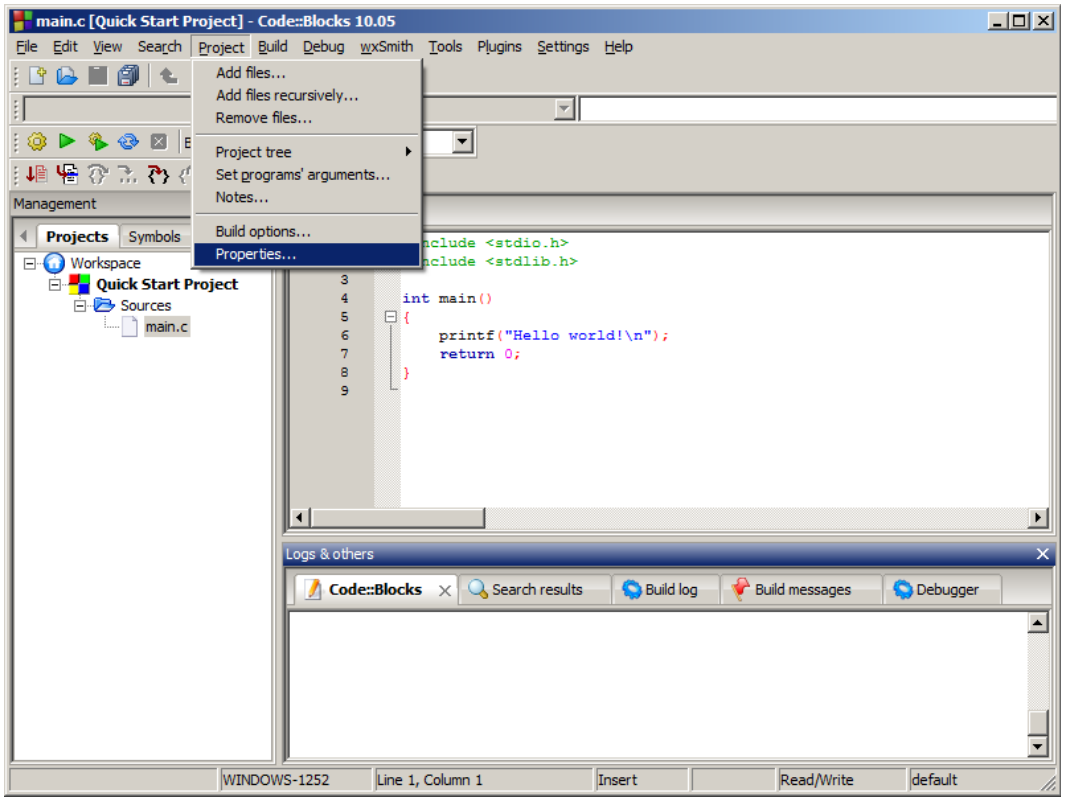
Once written, the program should compile and link fine. The last consideration to make is to ensure the *threespace_API.dll* file is in the correct folder. Proper locations for the .dll are either:

1. A folder listed in the operating system "path".
2. The same directory as the built binary program created by Visual Studio.

With this final consideration made, programs may now be written and executed that interact with the YEI 3-Space family of sensors. Included with the API is the test source file *threespace_test.cpp*. Import this source file into an empty Visual Studio project and follow the steps detailed above to integrate the 3-Space API. Once done, build and run the program to ensure the project is configured correctly.

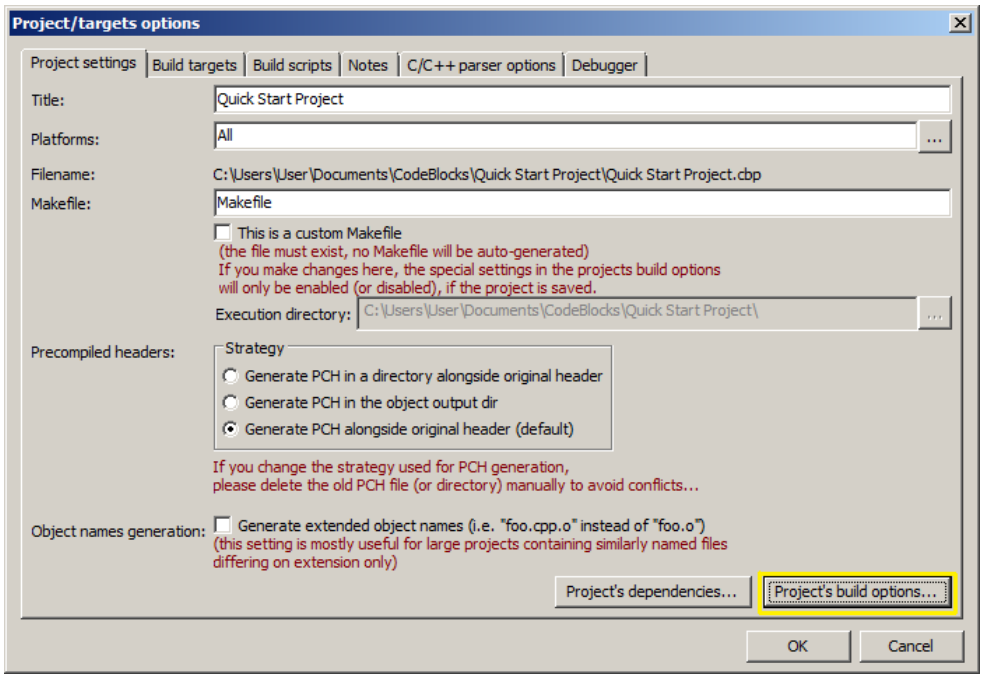
CodeBlocks

Assuming a new project has been created, the first step is to go to the project's properties. Access to the project's properties can be found by clicking the “Project” menu from the taskbar at the top of the window and selecting “Properties...”.

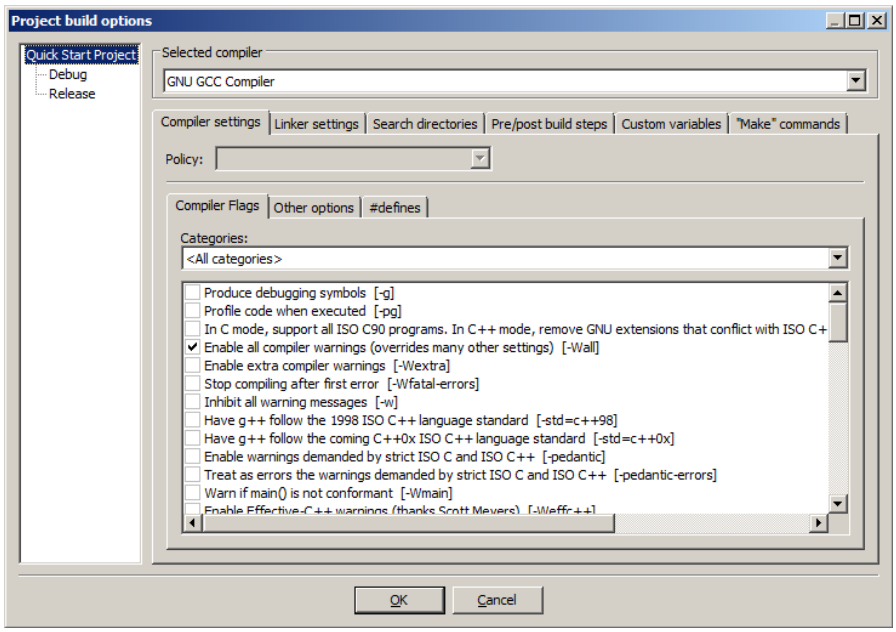


The main editor view of CodeBlocks. Note how the “Properties...” item in the “Project” menu is highlighted.

Once the Project's properties menu item has been selected, a new prompt titled “Project/targets options” will be displayed. From here, click on the “Project build options...” button found in the lower-right area of the prompt. This will bring up a new prompt titled “Project build options”.



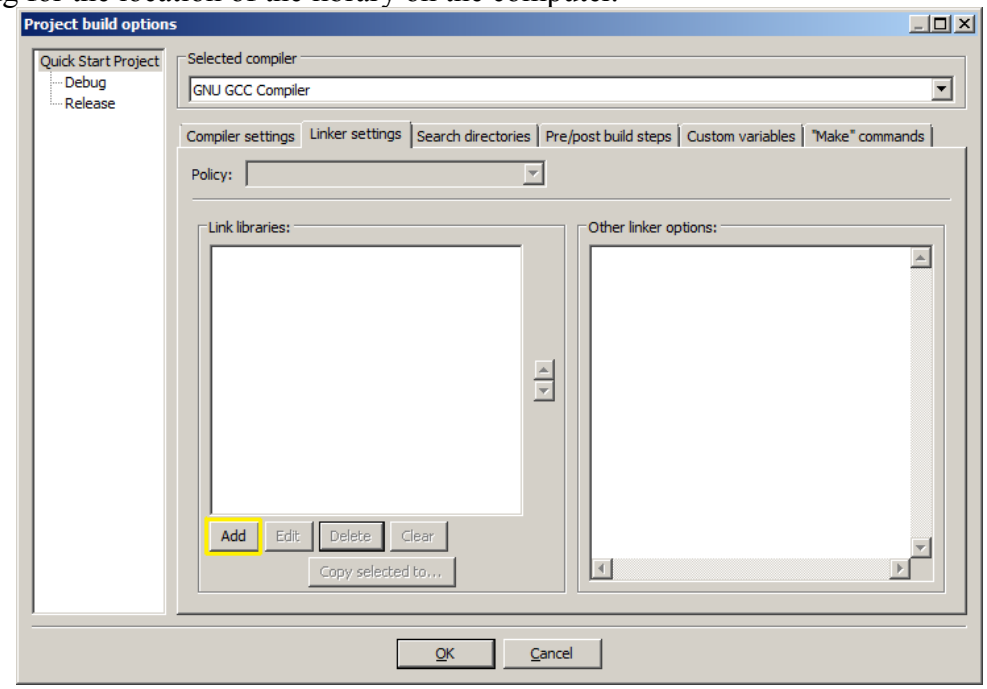
The “Project/targets options” prompt with the “Project's build options...” button highlighted.



The “Project build options” prompt.

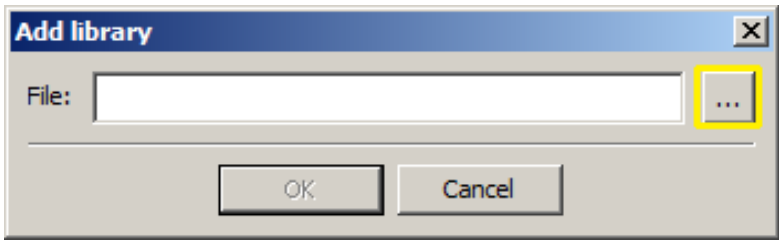
With the “Project build options” prompt is open, the next step is to tell CodeBlocks what dynamic link library to link to during the program's runtime. To do this first click on the “Linker settings” tab of the prompt. Once clicked, two empty areas will be presented in the prompt. The left area is the area of

interest. This area will list all additional libraries to link. Click the “Add” button to open another prompt asking for the location of the library on the computer.

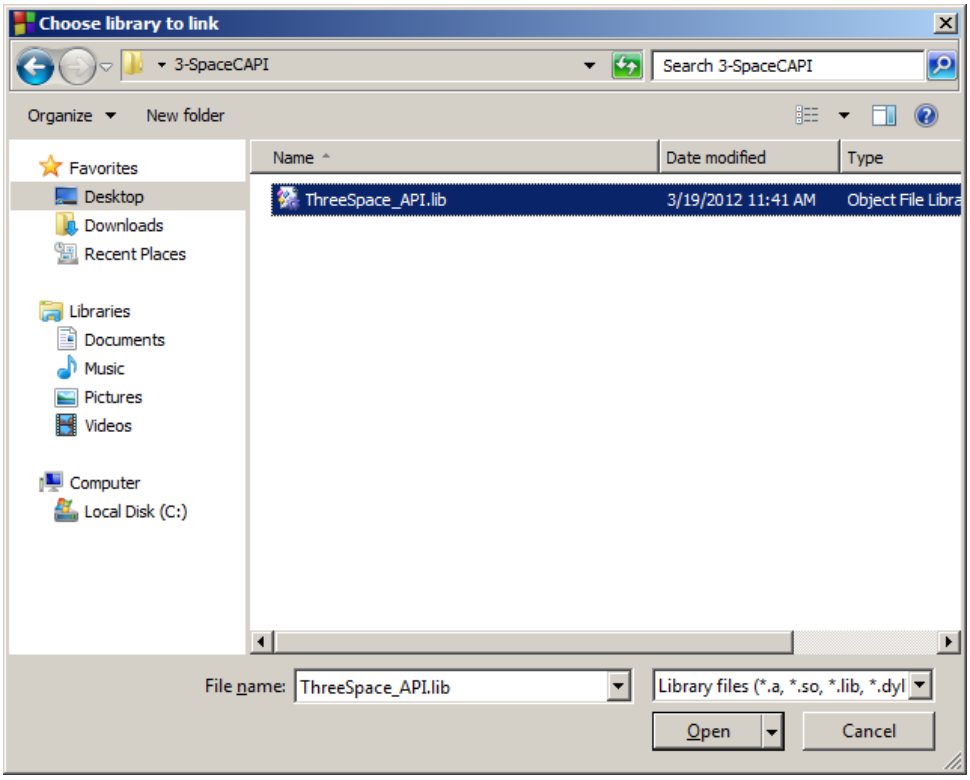


The “Project build options” prompt after the “Linker settings” tab has been clicked. Please note how the “Add” button is highlighted.

From here, click the “...” button to open a file hierarchy browser prompt. Use this prompt to search the computer for the folder the C API was downloaded to (or location that *ThreeSpace_API.lib* was stored). Select the *ThreeSpace_API.lib* file from this folder and click “Open”.



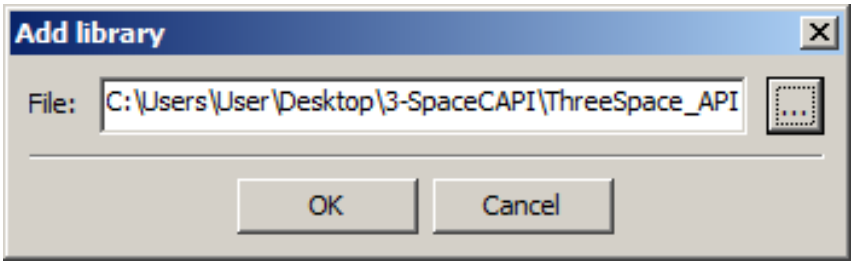
The prompt asking for the location of the library. Please note that the “...” button is highlighted.



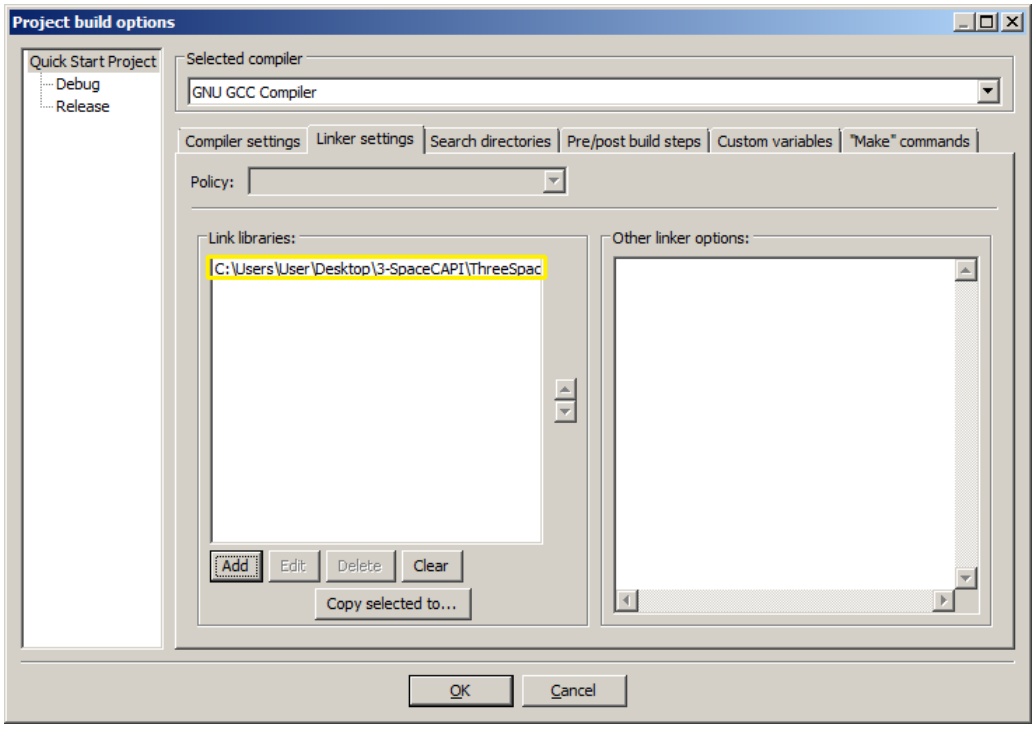
The file hierarchy browser prompt open. It is currently showing the location of the *ThreeSpace_API.lib*.

Once found, CodeBlocks will display a prompt asking “Keep this as a relative path?”.
If the API is stored in a location on the computer where multiple projects intend to access it and the API's location on the computer will never change select “No”.
If the API's location on the computer is dependent on the location of the CodeBlocks project's folder hierarchy and its location will be the same RELATIVE to the project's folder select “Yes”.
If it is unknown which to choose, select “No” and just don't move the API files around on the computer during the lifetime of the CodeBlocks project.

Once this choice has been made the file path to the library will be displayed in the textbox of the prompt. Click “OK” to add this path. Once “OK” is pressed, the new path to the library will be seen in the left area of the “Linker settings” tab area.

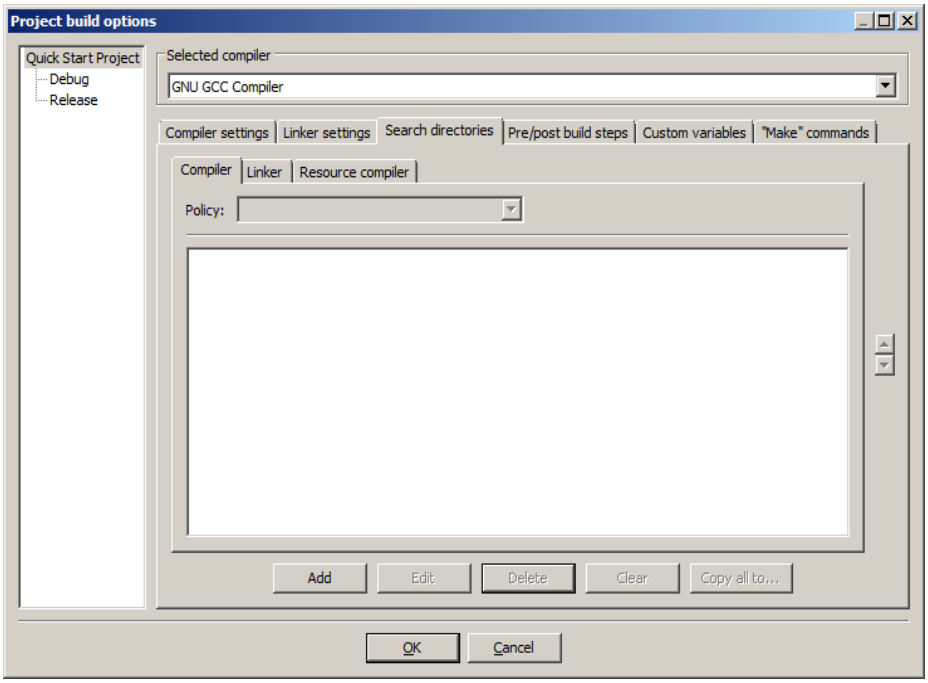


The prompt asking for the location of the library. This time the path to our library is in the textbox.

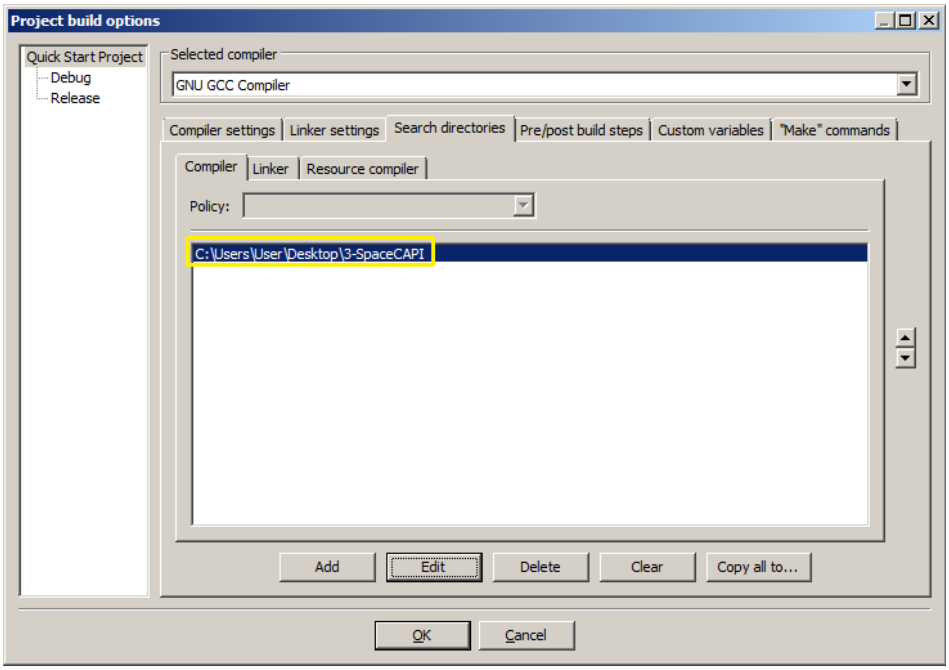


The “Project build options” prompt with our new path highlighted.

Once the path to the library has been added, the next step is to tell CodeBlocks where to find the header for the API at (*threespace_api.h*). To do this, click on the “Search directories” tab of the “Project build options” prompt. Once clicked, ensure that the “Compiler” tab is selected from the sub-area created by the “Search directories” tab. With the “Compiler” tab selected, a large empty area along with various buttons will be shown. The large empty area will list all the paths CodeBlocks will search through when compiling code. From this view, click the “Add” button near the bottom of the prompt. Like when adding a library a prompt will be displayed asking for a directory to add. Follow a similar procedure to adding a library. Click the “...” button, find the folder containing *threespace_api.h*, click “OK”, decide whether to keep the path as “relative” or not, and click “OK” again. Now the new path can be seen on the prompt.



The "Project build options" prompt with the "Search directories" tab selected and the "Compiler" tab selected inside of the "Search directories" tab.



The same view as above, but after the new directory has been added. The new directory has been highlighted.

Once this is done, click “OK” to close the “Project build options” prompt. Press “OK” again to close the “Project/targets options” prompt.

The only additional step to take now is to add the *ThreeSpace_API.dll* to the proper directory so the operating system may find it when the utilizing application runs. There are a couple different locations that the .dll may be copied to:

1. A folder listed in the operating system “path”.
2. The same directory as the built binary program created by CodeBlocks.

Once copied, include the header to the current project by writing the following line to the top of the appropriate source file:

```
#include "threespace_api.h"
```

With this final consideration made, programs may now be written and executed that interact with the YEI 3-Space family of sensors. Included with the API is the test source file *threespace_test.cpp*. Import this source file into an empty CodeBlocks project and follow the steps detailed above to integrate the 3-Space API. Once done, build and run the program to ensure the project is configured correctly.

Details about more advanced features of the discussed functions as well as the full feature set of the 3-Space API is included in the API documentation. Please reference these documents for mastering the 3-Space C API.



YEI Technology
630 Second Street
Portsmouth, Ohio 45662

Toll-Free: 888-395-9029
Phone: 740-355-9029

www.YeiTechnology.com
www.3SpaceSensor.com

Patents Pending
©2007-2012 Yost Engineering Inc.
Printed in USA